MEETING ACTION ITEM EXTRACTOR USING NLP

Submitted in partial fulfillment of the requirements of the

Mini-Project of Natural Language Processing Lab for

Fourth Year of

Bachelors of Engineering

by

Samiksha Patil- 231A044/39

Department of Artificial Intelligence & Data Science Engineering
Rizvi College of Engineering

University of Mumbai

2026-2027

CERTIFICATE

This is to certify that the mini-project entitled “Meeting Action Item Extractor Using NLP” is a bonafide work of Samiksha Patil submitted to the University of Mumbai in partial fulfillment of the requirement for the Mini-Project of Natural Language Processing Lab for Fourth Year of the Bachelor of Engineering in “Artificial Intelligence & Data Science”.

_____

Guide

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Samiksha Patil

Date: 24-10-2026

ABSTRACT

Meetings contain valuable information, but important tasks, responsibilities, priorities, and deadlines are often hidden inside long and unstructured discussions. The Meeting Action Item Extractor Using NLP is a Streamlit-based application designed to automatically convert meeting transcripts and uploaded documents into structured action items. The system accepts pasted meeting text and multiple document formats, extracts candidate action statements, uses a locally hosted Llama 3.2:3b model through Ollama to interpret task information, and applies deterministic validation and normalization rules to improve reliability. For every identified task, the system attempts to determine the responsible person, action item, original deadline phrase, normalized calendar deadline, and priority. The application also generates a concise meeting summary, provides workload and priority analytics, and supports CSV export of the extracted results.

The project combines Natural Language Processing concepts such as sentence segmentation, action detection, named-entity/person extraction, regular-expression-based deadline recognition, rule-based information extraction, normalization, and text summarization with a local Large Language Model. A major design objective is to avoid treating discussion or completed activities as future action items. The system therefore uses deterministic safety checks, fallback extraction, priority rules, and near-duplicate detection around the LLM output.

Index

Sr. No	Title	Page No
1.	INTRODUCTION	1
2.	SYSTEM ANALYSIS	3
3.	SYSTEM DESIGN	5
4.	TECHNOLOGIES USED	7
5.	IMPLEMENTATION	9
6.	OUTPUT SCREENSHOTS	11
7.	TESTING	14
8.	RESULTS AND DISCUSSIONS	15
9.	CONCLUSION	16
10.	REFERENCES	17
11.	APPENDIX	18

1. INTRODUCTION

1.1 Background

Meetings are a common part of academic, business, software-development, project-management, and organizational workflows. During a meeting, participants discuss issues, make decisions, assign responsibilities, and agree on deadlines. However, the information is usually expressed in natural language rather than in a structured task-management format. A person reading the complete transcript must manually identify statements such as “Rahul will prepare the report by Friday” and convert them into a task, owner, and deadline. Natural Language Processing (NLP) provides techniques for converting unstructured human language into machine-readable information. The proposed project applies NLP and a local language model to identify action-oriented statements from meeting text and organize them into a structured table.

1.2 Problem Statement

In most organizations, meetings generate long and unstructured discussions where important tasks, owners, and deadlines can be buried in conversation. Manual review of transcripts is time-consuming and can lead to missed tasks or incorrect deadlines. The project addresses this problem by developing an application that automatically extracts actionable tasks, identifies their assignees, detects deadlines, normalizes relative dates, assigns priority, generates a meeting summary, and provides downloadable results.

1.3 Objectives

- Automatically identify genuine action items from meeting transcripts.
- Extract the person responsible for each action item when a person is stated or can be safely inferred.
- Detect both relative and absolute deadline expressions.
- Normalize expressions such as “tomorrow”, “next week”, weekdays, and explicit dates using the selected meeting date.
- Classify tasks into High, Medium, or Low priority using deterministic rules.
- Reduce duplicate tasks when multiple input sources contain repeated content.
- Generate a concise summary from the structured action-item results.
- Provide a simple web interface for input, review, analytics, and CSV export.

1.4 Scope

The current system focuses on English-language meeting text and document-based inputs. It supports pasted text and multiple common file formats through the Streamlit interface. The project does not perform live audio transcription and does not directly integrate with Zoom, Microsoft Teams, or other meeting platforms.

Included	Not Included
Meeting transcript / notes processing	Live microphone or audio transcription
PDF, TXT, MD, CSV, JSON, DOCX, XLSX/XLS and PPTX inputs	Real-time meeting-platform integration
Action, assignee, deadline and priority extraction	Guaranteed understanding of implicit commitments
Relative deadline normalization	Multilingual extraction
Summary, analytics and CSV export	Enterprise task-management integration

1.5 Significance

The project reduces the amount of manual work required after meetings. A structured list of action items makes ownership and deadlines easier to review, while the analytics section gives a quick overview of workload and priority. The use of a local LLM also provides a practical approach for experimenting with AI-assisted extraction without requiring every meeting transcript to be sent to an external hosted model.

2. SYSTEM ANALYSIS

2.1 Existing System

In a conventional workflow, meeting participants or project coordinators read a transcript or meeting notes and manually identify tasks. They then copy each task into a spreadsheet or task-management system and separately calculate deadlines and priorities. This approach is repetitive, slow for long transcripts, and vulnerable to omissions.

2.2 Proposed System

The proposed system accepts meeting content through a Streamlit interface. Files are converted into text, the text is split into manageable sentence-level units, and candidate action sentences are passed to the local Llama 3.2:3b model through Ollama. The returned structured information is validated using deterministic rules. Deadline expressions are normalized against the meeting date, priorities are assigned using rule-based logic, and duplicate tasks are removed before results are shown to the user.

2.3 Functional Requirements

- The system shall accept manually pasted meeting text.
- The system shall accept supported document files through a file uploader.
- The system shall extract readable text from uploaded sources.
- The system shall identify candidate action-item sentences.
- The system shall extract action description and assignee information.
- The system shall detect and normalize deadlines.
- The system shall classify priority as High, Medium, or Low.
- The system shall remove duplicate or near-duplicate action items.
- The system shall display extracted tasks in a structured table.
- The system shall generate a meeting summary.
- The system shall display action-item and priority analytics.
- The system shall allow the result table to be downloaded as CSV.

2.4 Non-Functional Requirements

Requirement	Description
Usability	The interface should be simple enough for a user to paste/upload content and obtain results with minimal steps.
Reliability	Invalid model output and non-action sentences should be rejected rather than displayed as tasks.
Maintainability	Extraction logic is separated into app, AI extraction, and deterministic extraction modules.
Performance	Sentence-level processing reduces the amount of text handled by the local model at one time.
Privacy	The primary AI extraction path uses a locally hosted Ollama model.
Portability	The application can be executed locally using Python and Streamlit.

2.5 Feasibility Analysis

Feasibility	Analysis
Technical	Python, Streamlit, Pandas, Plotly, spaCy and Ollama provide the required development capabilities.
Operational	The workflow is simple: provide meeting content, extract tasks, review, and export.
Economic	The local LLM approach can be used without per-request cloud inference charges.
Schedule	The project can be developed incrementally using deterministic NLP first and AI extraction as an enhancement.

3. SYSTEM DESIGN

3.1 Overall Architecture

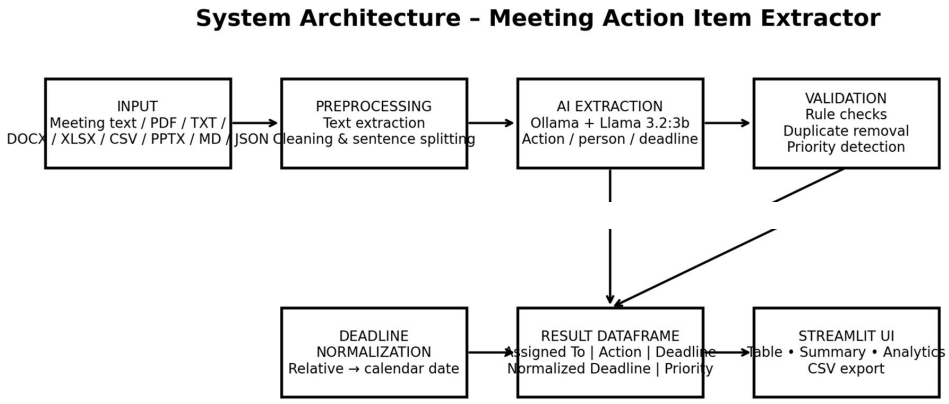


Figure 3.1: High-level architecture of the Meeting Action Item Extractor.

3.2 System Workflow

- User selects the meeting date and either pastes meeting text or uploads one or more supported files.
- The application extracts text from each source and combines the readable content.
- The text is divided into sentence-level units.
- A deterministic candidate gate removes obvious discussion/status sentences and retains sentences that look like assignments, imperatives, or deadline-driven tasks.
- Ollama runs the Llama 3.2:3b model on each candidate sentence and returns structured JSON containing action, assignee, deadline and a model decision.
- The application validates the model output and applies deterministic safety checks.
- Deadlines are normalized using the selected meeting date.
- Priority is calculated using deterministic rules so that the small local model does not arbitrarily assign priority.
- Near-duplicate tasks are removed while preserving the more informative record.
- The final Pandas DataFrame is displayed along with a summary, analytics, and CSV download.

3.3 Input and Output Data Model

Field	Purpose	Example
Assigned To	Person responsible for the task	Rahul
Action Item	Normalized task description	prepare final report
Deadline	Original deadline phrase	next Friday
Normalized Deadline	Calendar-formatted deadline	23 October 2026
Priority	Task urgency classification	High

3.4 Deadline Normalization Design

The system treats the meeting date as the reference point for relative deadlines. It supports expressions such as today, tomorrow, tonight, weekdays, next weekday, next week, this week, next month, this month, in a number of days/weeks/months, and explicit dates such as “25 September”. This prevents a phrase such as “next week” from being interpreted relative to the current computer date instead of the date of the meeting.

3.5 Priority Design

Priority is determined using explicit urgency phrases and deadline proximity. High-priority indicators include terms such as urgent, immediately, ASAP, critical, blocker, emergency, security issue, and must. Near-term deadlines can also increase priority. Low-priority indicators include optional, whenever, eventually, later, non-urgent, and nice-to-have. Other valid tasks default to Medium.

3.6 Duplicate Handling

When several input files or text sources contain the same task, the AI extraction layer normalizes action wording and compares assignee, deadline, string similarity, token overlap, and subset relationships. This helps prevent repeated rows such as “prepare report” and “prepare the project report” from being shown as separate tasks when they represent the same assignment.

4. TECHNOLOGIES USED

4.1 Programming Language

Python is used as the primary development language because it provides mature NLP, data-processing, document-processing, and web-application libraries.

4.2 NLP and AI Components

Technology	Role in the Project
spaCy	Used by the deterministic extractor for NLP processing such as POS-based action detection and person/entity processing.
Ollama	Runs the local language model and provides the interface used by the application to request structured extraction.
Llama 3.2:3b	Local LLM used to interpret candidate meeting sentences and return structured action-item information.
Regular Expressions (Regex)	Used for deadline patterns, assignment patterns, validation and text cleaning.
Rule-based NLP	Validates model output, detects priority, normalizes dates and handles fallback cases.

4.3 Data and Visualization Libraries

Technology	Role
Pandas	Stores and manipulates extracted action-item results in tabular form.
Plotly	Creates interactive action-items-by-person and priority-distribution charts.
Python datetime/calendar	Supports deadline calculations and date normalization.

4.4 Web Interface

Streamlit is used to build the user interface. The application provides a meeting-date selector, text area, multi-file uploader, extraction button, results table, CSV download, meeting summary, metric cards, and visual analytics.

4.5 Document Processing

The current application is designed to read multiple file formats. Text-like files are decoded directly; structured files are read through appropriate Python libraries; PDFs are processed using pypdf; and document/spreadsheet/presentation inputs are converted into text before AI extraction.

4.6 Project Files

File	Purpose
app.py	Streamlit user interface, file handling, result display, summary, analytics and CSV export.
ai_extractor.py	Local LLM extraction, validation, duplicate removal and AI-assisted structured processing.
extractor.py	Deterministic NLP/fallback extraction, deadline normalization, priority logic and summary generation.
extractor_backup.py	Backup version of the extraction logic.
requirements.txt	Python package requirements used to set up the environment.
.env / .env.example	Environment configuration files; secrets should remain outside source control.
.gitignore	Prevents selected local/configuration files from being committed.

5. IMPLEMENTATION

5.1 Application Initialization

The application starts as a Streamlit web application. The user selects a meeting date, enters meeting text, or uploads one or more supported files. The selected meeting date is passed to the extraction layer so that relative expressions such as tomorrow, Friday, or next week are resolved using the meeting date.

5.2 File Input and Text Extraction

The file-processing function identifies the uploaded file extension and converts the source into text. The interface accepts PDF, TXT, Markdown, CSV, JSON, DOCX, XLSX/XLS and PPTX inputs. Multiple files can be processed in the same request, and readable text from the sources is combined before extraction.

5.3 Candidate Sentence Detection

Before calling the local model, the AI extractor performs a deterministic candidate check. Sentences that are too short, meeting titles, status statements, or completed activities are rejected. Sentences containing assignment language, imperative action verbs, or an action combined with a deadline are retained as candidates.

5.4 Local AI Extraction

Each candidate sentence is sent to the Llama 3.2:3b model through Ollama. The prompt instructs the model to return JSON with the fields is_action_item, assigned_to, action_item, deadline and priority. The prompt also explicitly instructs the model not to invent names or deadlines and not to convert past activities into future tasks.

5.5 Deterministic Validation

The application does not blindly trust the LLM output. The returned data is validated, action text is cleaned, empty or invalid assignees are normalized, deadlines are standardized, and priority is recalculated using deterministic rules. If the model fails or incorrectly rejects a passive-voice task, a deterministic fallback can recover supported patterns.

5.6 Deadline Normalization

The deadline module converts relative phrases to actual dates using the meeting date. For example, “tomorrow” becomes one day after the meeting date, “next week” becomes seven days after the meeting date, and explicit dates are parsed into calendar values.

5.7 Result Presentation

The final output is stored in a Pandas DataFrame. The interface displays Assigned To, Action Item, Deadline, Normalized Deadline and Priority. Priority labels are visually represented as High, Medium and Low, and the user can download the complete result as a CSV file.

5.8 Summary and Analytics

A concise summary is generated from the structured result rather than from unsupported assumptions about the transcript. The analytics section calculates total action items, number of assigned people, tasks with deadlines, tasks without deadlines, and the number of high-priority tasks. Two visualizations show action-item counts by person and priority distribution.

5.9 Key Implementation Logic

Module	Main Responsibilities
app.py	Input collection → file text extraction → AI processing → table → summary → analytics → CSV
ai_extractor.py	Candidate filtering → Ollama → JSON parsing → validation → deadline/priority logic → deduplication
extractor.py	Regex/POS fallback → assignee/action/deadline extraction → normalization → priority → summary

6. OUTPUT SCREENSHOTS

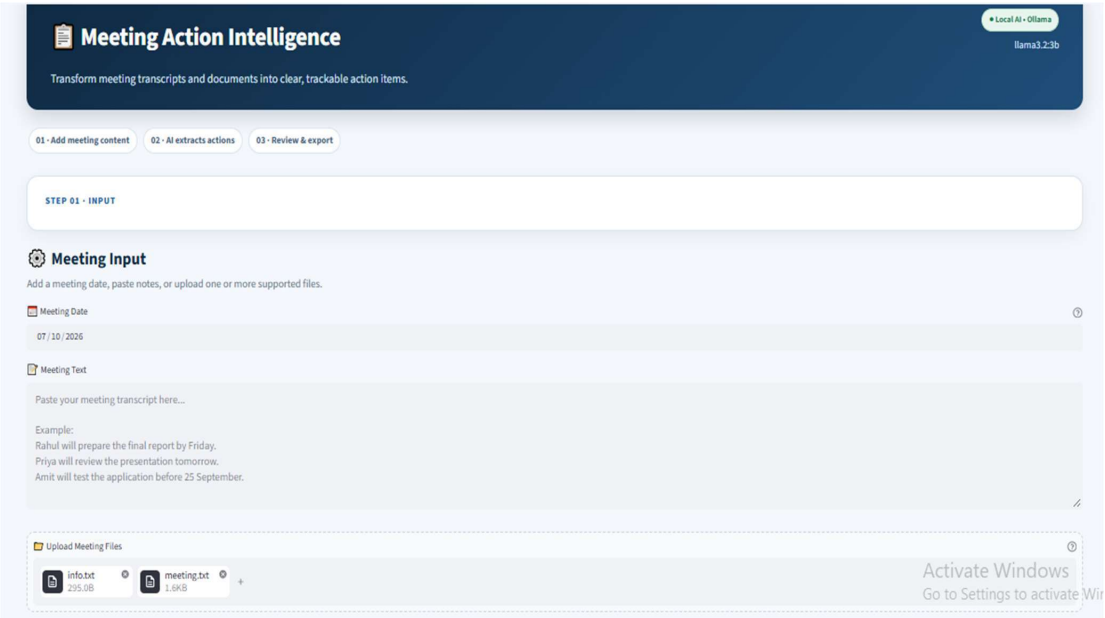


Figure 6.1 – Application Home / Meeting Input

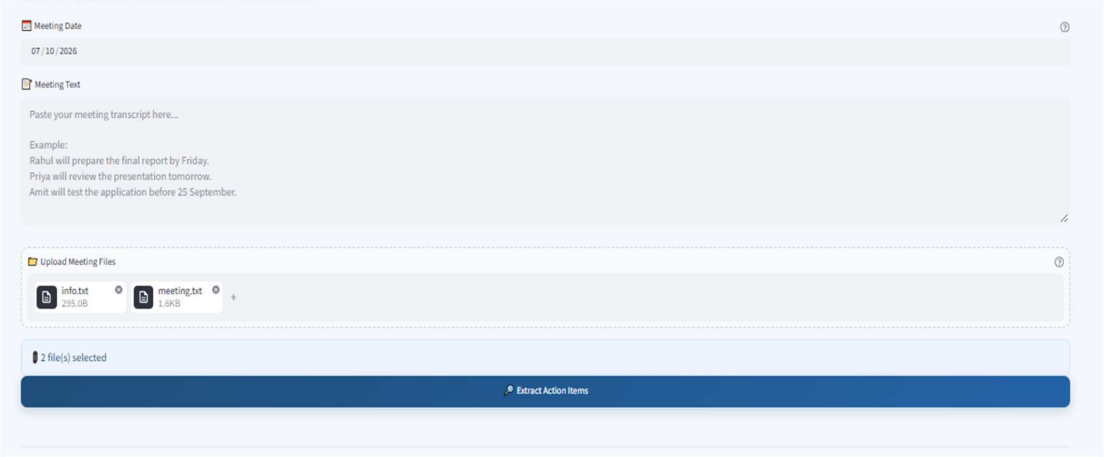


Figure 6.2 – Uploaded Meeting File

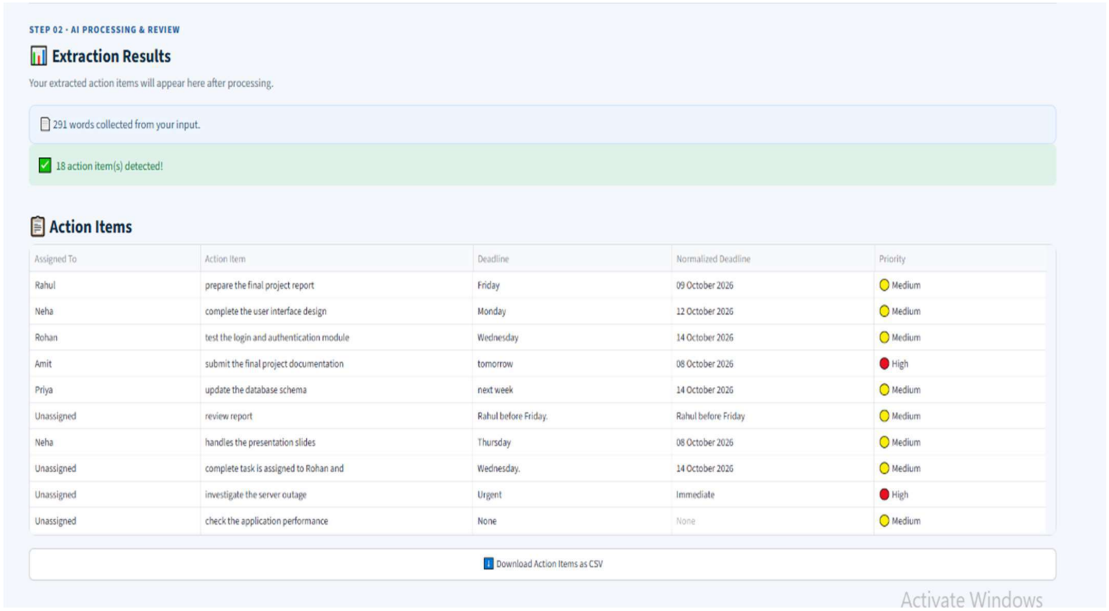


Figure 6.3 – Extracted Action Items

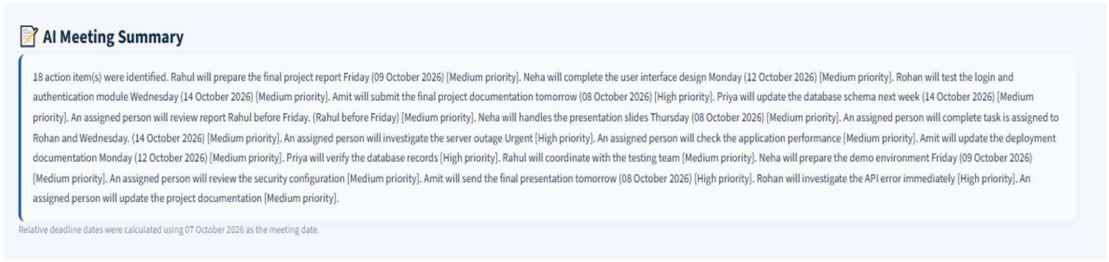


Figure 6.4 – AI Meeting Summary

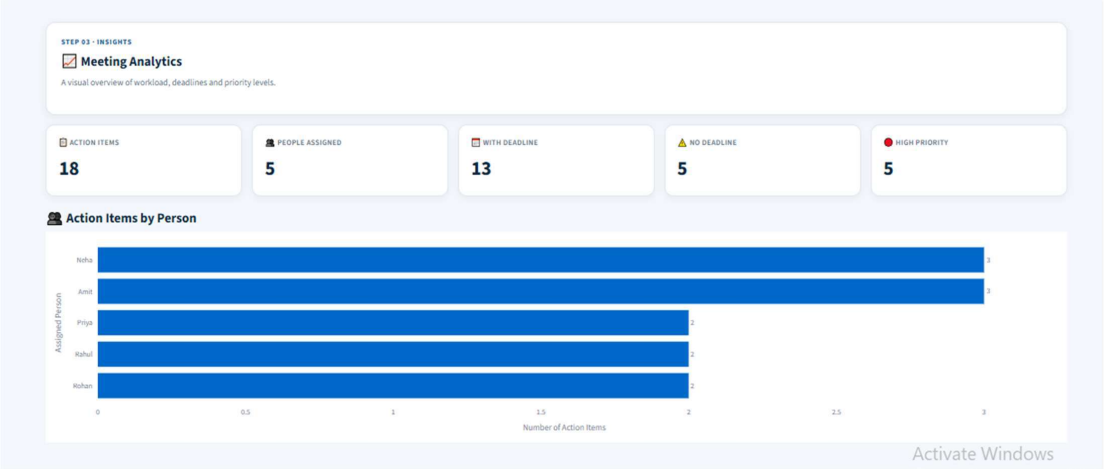


Figure 6.5 – Meeting Analytics

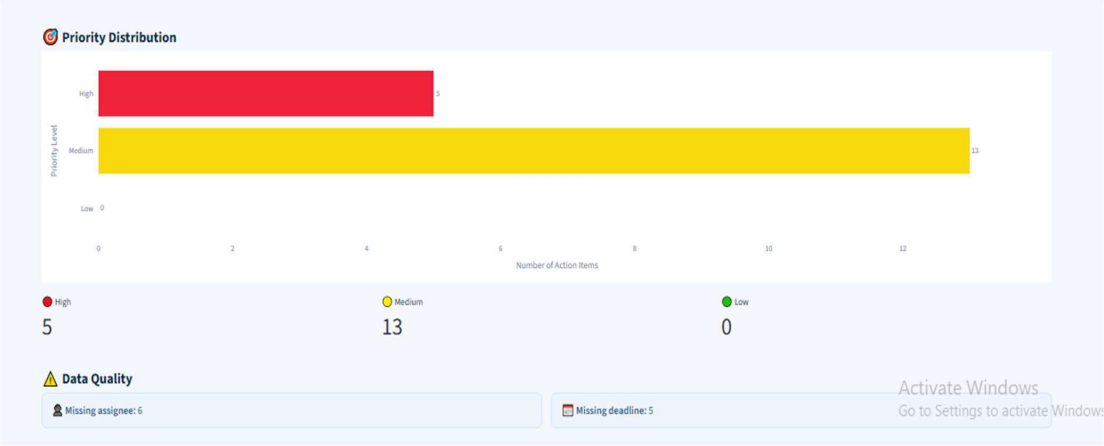


Figure 6.6 – Priority Distribution

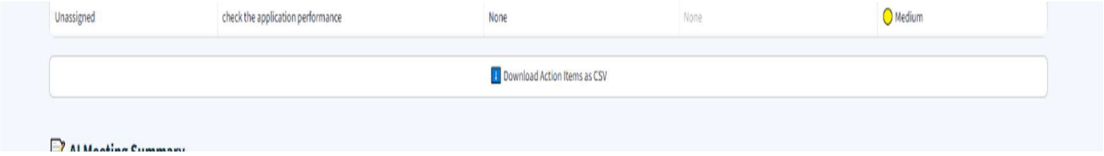


Figure 6.7 – CSV Export

7. TESTING

7.1 Testing Strategy

Testing focuses on functional behavior rather than claiming a numerical NLP accuracy that has not been measured on a labeled benchmark dataset. The system should be tested with normal assignments, relative deadlines, explicit dates, passive voice, discussion statements, completed statements, missing deadlines, duplicate sources, and invalid/empty input.

ID	Test Case	Input / Condition	Expected Result	Status
TC01	Paste valid action sentence	Rahul will prepare the report by Friday.	Task extracted with Rahul and Friday deadline	Pass
TC02	Relative deadline	Priya will review the presentation tomorrow.	Deadline normalized using meeting date	Pass
TC03	Next-week logic	Amit will submit the document next week.	Normalized date is calculated from meeting date	Pass
TC04	Absolute date	Neha will test the application before 25 September.	25 September detected and normalized	Pass
TC05	Discussion statement	The team discussed the current project status.	No action item created	Pass
TC06	Completed activity	The report was submitted yesterday.	No future action item created	Pass
TC07	Urgent task	Rahul must fix the security issue ASAP.	High priority assigned	Pass
TC08	Low priority	Review the optional documentation later.	Low priority assigned	Pass
TC09	Multiple files	Same task appears in two uploaded sources.	Near-duplicate task removed	Pass
TC10	Empty input	No meeting text or readable file.	Warning shown and extraction stopped	Pass
TC11	CSV export	Valid extracted result	CSV download is available	Pass
TC12	Analytics	Valid result with multiple priorities	Metrics and charts displayed	Pass

7.2 Limitations of Testing

The project currently does not include a large manually labeled benchmark dataset. Therefore, precision, recall, F1-score and extraction accuracy should not be presented as measured numerical results unless a labeled evaluation set is created and tested. For the academic report, functional testing and representative examples can be reported honestly, while quantitative NLP evaluation can be proposed as future work.

8. RESULTS AND DISCUSSIONS

8.1 Results

The implemented application provides an end-to-end workflow for converting meeting text into structured action items. The final interface supports direct text input as well as uploaded documents, processes candidate sentences through a local LLM and deterministic validation layer, and displays the extracted results in a tabular form.

- Action items are represented with an assigned person, task description, deadline and priority.
- Relative deadline expressions are normalized using the meeting date selected by the user.
- Discussion and completed-activity statements are filtered to reduce false action items.
- The AI layer is protected by deterministic validation and fallback extraction.
- Duplicate and near-duplicate tasks can be reduced when multiple sources contain repeated content.
- The user can review summary information and visual analytics before exporting the results.

8.2 Discussion

The combination of a local LLM and deterministic NLP is useful for this project because meeting language can be expressed in many different ways. A pure rule-based approach is predictable but may struggle with variations in sentence structure. A pure small LLM approach is flexible but may hallucinate fields or incorrectly classify status statements. The hybrid design attempts to combine the strengths of both: the LLM interprets the natural-language sentence, while deterministic rules decide whether the result is safe and consistent enough to display.

The system is especially useful when deadlines are expressed relatively. The meeting date acts as a stable reference point, allowing expressions such as “tomorrow”, “next Friday” and “next week” to be converted into calendar dates. This is important because using the computer's current date would produce incorrect deadlines for historical meetings.

8.3 Practical Applications

- Project management – convert meeting discussions into assignable tasks.
- Software development – capture development, testing, deployment and bug-fixing tasks.
- HR and operations – summarize follow-up activities from internal meetings.
- Consulting and client meetings – prepare structured action lists from meeting notes.

8.4 Current Limitations

- The system is primarily designed for English-language content.
- Accuracy depends on transcript quality and clarity of assignments.
- Implicit commitments without clear action language can be difficult to detect.
- No numerical accuracy benchmark is included unless a manually labeled evaluation dataset is created.
- Live audio transcription and direct meeting-platform integration are outside the current scope.

9. CONCLUSION

The Meeting Action Item Extractor Using NLP demonstrates how Natural Language Processing and local generative AI can be combined to automate a common post-meeting activity. The system transforms unstructured meeting content into structured action items containing responsibility, task description, deadline, normalized date and priority.

The project goes beyond simple keyword matching by using Llama 3.2:3b through Ollama for sentence-level interpretation and deterministic validation for reliability. The application also includes file ingestion, duplicate handling, deadline normalization, summary generation, analytics and CSV export, making it suitable as a practical prototype for project-management workflows.

Future improvements can include a larger labeled evaluation dataset, multilingual support, speaker diarization, live speech-to-text, integration with Zoom or Microsoft Teams, direct export to task-management platforms, calendar synchronization, user authentication, and stronger domain-specific models.

10. REFERENCES

1. S. Muppidi, J. Kandi, B. S. Kondaka, C. Kethireddy, and S. E. Kandregula, “Automatic meeting minutes generation using Natural Language processing,” in *2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT)*, 2023, doi: 10.1109/EASCT59475.2023.10393102.
2. W. Xiang and B. Wang, “A survey of event extraction from text,” *IEEE Access*, vol. 7, pp. 173111–173137, 2019, doi: 10.1109/ACCESS.2019.2956831.
3. Jurafsky, D. and Martin, J. H., Speech and Language Processing, 3rd ed. draft, Pearson / Stanford University.
4. spaCy Documentation, Natural Language Processing in Python, Explosion AI.
5. Streamlit Documentation, Python framework for building data and AI applications.
6. Pandas Documentation, Python Data Analysis Library.
7. Plotly Python Documentation, Interactive Data Visualization Library.
8. Ollama Documentation, Local Large Language Model Runtime.
9. Python Documentation, datetime, re, calendar and standard library modules.
10. PyPDF Documentation / pypdf, PDF text extraction library.
11. Project source code: app.py, ai_extractor.py, extractor.py and requirements.txt.

11. APPENDIX

A. Sample Meeting Input

Rahul will prepare the final project report by Friday.

Priya should review the presentation tomorrow.

Amit will test the application before 25 September.

Neha must fix the security issue ASAP.

The team discussed the current project status.

The previous report was submitted yesterday.

B. Expected Structured Output

Assigned To	Action Item	Deadline	Normalized Deadline	Priority
Rahul	prepare final project report	Friday	Meeting-date dependent	Medium/High
Priya	review presentation	tomorrow	Meeting-date + 1 day	High
Amit	test application	25 September	25 September	Medium
Neha	fix security issue	ASAP	ASAP	High

The exact normalized calendar dates depend on the meeting date selected by the user.

C. Key Source-Code Functions

File	Function / Component	Purpose
app.py	extract_file_text()	Converts uploaded files into readable text.
app.py	extract_action_items_with_ai()	Starts AI-assisted extraction from collected sources.
ai_extractor.py	_looks_like_action_candidate()	Filters sentences before LLM processing.
ai_extractor.py	_ollama_extract()	Calls Llama 3.2:3b through Ollama and requests JSON.
ai_extractor.py	_validate_item()	Validates and normalizes model output.
ai_extractor.py	_deduplicate()	Removes exact and near-duplicate tasks.
extractor.py	normalize_deadline()	Converts relative deadline phrases to dates.
extractor.py	detect_priority()	Assigns High, Medium or Low priority.
extractor.py	generate_summary()	Builds a concise summary from extracted tasks.

D. Suggested Screenshot Checklist

- Home/Input screen
- Meeting date and pasted transcript
- File upload screen
- Extraction result table
- Normalized deadline examples
- AI meeting summary
- Action-items-by-person chart
- Priority distribution chart
- CSV download control
- Final complete application screen

E. Installation and Execution

- Create and activate a Python virtual environment.
- Install the packages listed in requirements.txt and any NLP/model runtime dependencies used by the final code.
- Install and start Ollama locally.
- Ensure the Llama 3.2:3b model is available to Ollama.
- Run the Streamlit application using the command: streamlit run app.py
- Open the local Streamlit URL shown in the terminal.